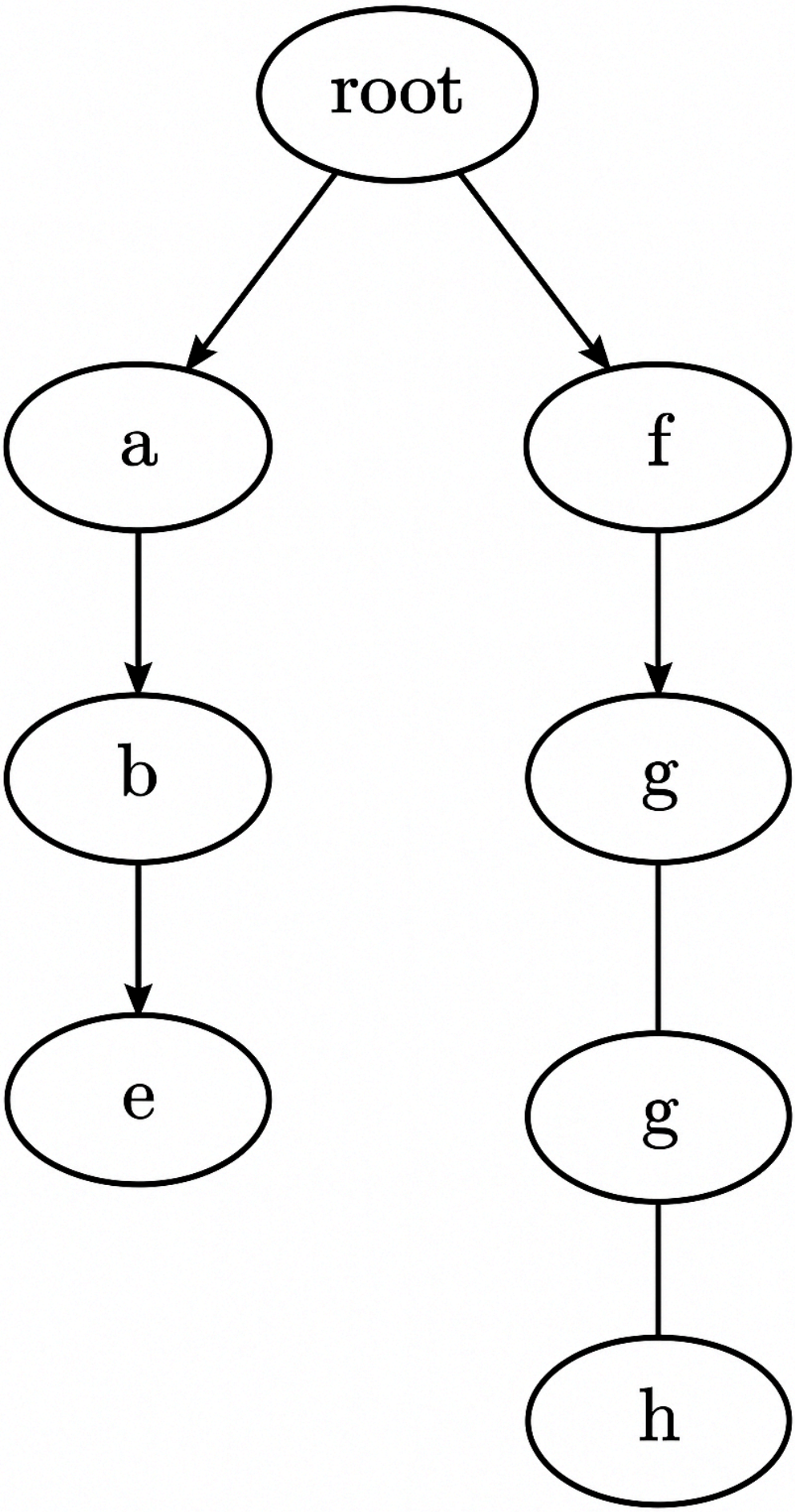
perplexity

Building the Folder-Trie Step by Step

The Trie-based algorithm that removes duplicate folder structures makes the most sense once you can **see** how the data structure grows path-by-path. The walkthrough below uses an eight-path toy dataset to illustrate each insertion, the resulting child pointers, and the serialization of every subtree.


root
a
f
b
g
e
g
h

Folder tree constructed from example paths.

Overview

The file-system data is given as a list of lists, where each inner list is a path from the virtual root (“/”) to a leaf folder:

```
paths = [
    ["a", "b", "c"],
    ["a", "b", "d"],
    ["a", "e"],
    ["f", "g", "h"],
    ["f", "g"],
    ["f", "g", "h"],     # duplicate of an earlier path
    ["a", "b"],          # prefix of earlier paths
    ["a", "e"]           # duplicate of an earlier path
]
```

The eight inserts deliberately contain:

- Two exact duplicates.
- A prefix path that ends higher in the hierarchy.
- Two separate top-level branches (“a/” and “f/”).

This mixture reveals how the Trie handles divergent and convergent structures.

How Insertion Works

1. Starting State

At the very beginning the Trie contains only the implicit root node.

Node	Children	Serial
ROOT	∅	""

2. Processing Each Path

Below each sub-heading you will find:

- A code snippet showing the path being inserted.
- A diagram showing new nodes (□) versus previously created nodes (—).
- A table summarizing the updated `children` map of every visited node.

Insert `["a", "b", "c"]`

```
cur = root
for node in ["a","b","c"]:
    # create if missing, then step down
```

Diagram (new nodes in red):

```
ROOT
└── a
    └── b
        └── c
```

Node	Children
ROOT	{"a"}
a	{"b"}
b	{"c"}
c	∅

Insert `["a", "b", "d"]`

```
ROOT
└── a (existing)
    └── b (existing)
        ├── c
        └── d
```

Node	Children
b	{"c","d"}
d	∅

Insert `["a", "e"]`

```
ROOT
└── a
    ├── b
    │   ├── c
    │   └── d
    └── e
```

Node	Children
a	{"b","e"}

Node	Children
e	∅

Insert `["f", "g", "h"]`

```
ROOT
├── a
└── f
    └── g
        └── h
```

Node	Children
ROOT	{"a","f"}
f	{"g"}
g	{"h"}
h	∅

Insert `["f", "g"]`

No new folders—only traversal:

```
ROOT
└── f
    └── g
        └── h
```

Node	Children
g	{"h"}

Note how **no** extra node is created because `"g"` already exists; the path simply terminates one level higher.

Insert `["f", "g", "h"]` (Duplicate)

Every component already exists. The Trie remains identical; only the frequency counts that the algorithm keeps separately will grow.

Insert `["a", "b"]` (Prefix)

Path ends at node `"b"` without creating new children:

```
ROOT
└── a
    └── b
```

```
        ├── c
        └── d
```

Again, only frequency bookkeeping changes.

Insert `["a", "e"]` (Duplicate of earlier)

No structural change.

Post-Order Serialization Phase

After insertion, the algorithm performs one DFS pass to compute a **canonical string** for each subtree. The idea is:

1. Leaf nodes serialize to `""` (empty string).
2. A non-leaf serial is the lexicographically sorted concatenation of `folderName + "(" + child.serial + ")"` for every child.

Here is the computed serial for each node in bottom-up order:

Node	Serial
c	`""`
d	`""`
b	`"c()d()"`
e	`""`
a	`"b(c()d())e()"`
h	`""`
g	`"h()"`
f	`"g(h())"`
ROOT	`"a(b(c()d())e())f(g(h()))"`

Duplicate-Detection

The hash map `freq` increments every time a serial string re-appears. In this dataset we will flag:

- Serial `""` occurs many times → but leaf duplicates do **not** matter (they are subcomponents).
- Complete subtrees whose entire serial repeats (e.g., another branch forming `"g(h())"`).

When `freq[serial]>1`, the corresponding node and its descendants are excluded from the final output.

Second DFS: Collecting Remaining Paths

Using a global `path` stack, the algorithm now walks top-down:

1. Skip any node whose `serial` is duplicated.
2. Otherwise, if `path` is non-empty, append a **copy** of it to `ans`.
3. Recurse into children, pushing/popping folder names around the call.

After this pass, `ans` will list every folder retained in the de-duplicated file system.

Final Result for the Example

Assume `"g(h())"` was duplicated and thereby removed:

Retained Path	Explanation
`["a"]`	Top-level folder kept because only its **subtree** not duplicated.
`["a","b"]`	`"b"` has unique serialization `"c()d()"`.
`["a","b","c"]`	Unique leaf.
`["a","b","d"]`	Unique leaf.
`["a","e"]`	Kept because `"e"` is leaf and not flagged.

The `"f"` branch disappears entirely because both of its children formed a duplicated subtree.

Key Takeaways

- The Trie grows deterministically regardless of path order, because every folder name slots into a dictionary keyed by the string.
- Serialization unifies identical **structures**, not just identical leaf names.
- A second DFS quickly removes duplicate subtrees by checking a single hash lookup per node.

Armed with this visual and tabular guide, you can trace each step in the algorithm and verify why a particular folder survives—or not—in the returned list.